

Quality-diversity for Synthesizer Sound Matching

NAOTAKE MASUDA^{1,a)} DAISUKE SAITO^{1,b)}

Received: September 30, 2022, Accepted: December 2, 2022

Abstract: Synthesizers have become commonplace in music production due to their ability to create a diverse and unique timbre. However, many users find it difficult to control a synthesizer due to the complexity of the relationship between synthesis parameters and output sound. As such, the *sound matching* task focuses on the estimation of synthesis parameters that recreate the target sound as closely as possible within the capabilities of the synthesizer. In this paper, we introduce a quality-diversity approach to the problem of sound matching by introducing a novelty objective to genetic algorithm-based sound matching. We show that this QD method allows for the discovery of a collection of matches that are more diverse compared to matches found by standard genetic algorithms. These matches are spread across a space of interpretable spectral features, making it useful for the intuitive discovery of unique and useful matches.

Keywords: Synthesizer, music production, genetic algorithm, quality-diversity algorithm

1. Introduction

Synthesizers have become an essential instrument in modern music production, and are often the centerpiece in electronic music. Synthesizers are unique in that they are able to produce a wide range of timbres. This is possible because synthesizers allow for direct control of the parameters of the synthesis algorithm, by either physical knobs and sliders in the case of analog synthesizers, or the GUI in the case of software synthesizers.

However, many users find it difficult to control a synthesizer as they intend, due to the complexity of the relationship between synthesis parameters and output sound. As such, there have been many works attempting to assist the user of a synthesizer. One way would be to map the synthesis parameters of a synthesizer to a more intuitive parameter. One can imagine that controlling a synthesizer using knobs labeled with descriptors such as “brightness” or “noisiness” would be more intuitive than interacting with the raw parameters. It is however difficult to entirely replace the synthesis parameters, as the diversity of synthesizable sounds cannot be fully explained in simple adjectives.

Another method for assisting the user is *sound matching*, i.e., the estimation of synthesis parameters that replicate a given sound as closely as possible using a certain synthesizer. The user can input a sound that is close to their intention as a query, and a sound matching system will estimate matching synthesis parameters. Then the user can tweak the sound further by interacting with the synthesis parameters.

It is also expected that the matches do not sound exactly like the original sound but instead have a unique timbre. This is of interest to many musicians, as synthesized imitations of a certain instrument (synth brass, synth string etc.) are often preferred to

the original instrument in some situations.

Many techniques have been employed for sound matching. One way is to build models that can estimate the synthesis parameters from audio input. For example, models such as linear regression [1] and neural networks [2], [3], [4] have been used. Another way is to iteratively optimize the synthesis parameters. In each iteration, the synthesis parameter is rendered to audio and the match quality is used to guide the generation of the next parameter. A popular method for iterative optimization is the genetic algorithm (GA). In GA, a population of synthesis parameters are evaluated and ones with the best match quality are mutated and crossed over to create the next generation.

While traditional GAs pursue to only maximize the fitnesses of the population, quality-diversity algorithms [5] have been proposed to find high-performing and diverse individuals. In an earlier work, the authors applied quality-diversity algorithms to the sound matching problem to find a diverse collection of matches [6]. The diversity of an individual is calculated as its sparsity in the space of behavioral characteristics (in this case, audio descriptors). By finding diverse matches for a sound, users can choose a variation that they prefer. Since the algorithm produces matches that are spread out in terms of certain audio descriptors, we can obtain a map of matches whose dimensions can be interpreted intuitively.

In this paper, we further evaluate the validity of this approach. We evaluate the final population of the evolution with multiple objective measures for match quality. We compare quality-diversity algorithms with a GA baseline and a neural network-based method. Also, we evaluate the match quality with subjective evaluation. With these results, we discuss the design of the behavioral characterization and its effect on the performance. Finally, we discuss alternative criteria for sound matching and discuss possible applications.

In Section 2, we cover the background of this work including

¹ The University of Tokyo, Bunkyo, Tokyo 113–8656, Japan

^{a)} n.masuda@gavo.t.u-tokyo.ac.jp

^{b)} dsk.saito@gavo.t.u-tokyo.ac.jp

the basics of quality-diversity algorithms and previous works in synthesizer sound matching. In Section 3, we explain the workflow of the proposed algorithm. The setup of the sound matching experiment, including details about the synthesizer, is explained in Section 4 and its results are shown in Section 5. We discuss our findings in Section 6 and end with a conclusion in Section 7.

2. Background

2.1 Evolutionary Algorithms

Evolutionary algorithms are a family of metaheuristic optimization algorithms that take inspirations from biological evolution. *Genetic algorithm* (GA) is the most basic type of evolutionary algorithm. In GA, an individual consists of a *genotype* and a *phenotype*. The genotype is the equivalent of the genomes of an organism and is usually expressed as an array of binary numbers in GA, but some GA methods deal with genotypes that are expressed as real numbers. In algorithms such as genetic programming, the genotype is expressed as a tree structure instead. The phenotype is the equivalent of the physical form of an organism. The fitness of an individual is evaluated by how well the phenotype performs against a certain problem.

In a typical GA, the first step is to randomly generate the initial population of individuals. Next, each individual is evaluated against the problem, and is assigned a fitness value. A subset of this population is then selected from the population and genetic operators are applied to the individuals of the subset to create the next generation. This process is repeated for a fixed number of generations, or an adequate solution has been found.

In *multi-objective GA* (MOGA), multiple objectives are optimized at once to find a set of *Pareto-optimal* solutions. A solution is Pareto-optimal when it is not dominated (made inferior in every objective) by any other solution.

2.1.1 Novelty Search

Traditional evolutionary algorithms have utilized the power of evolution to solve optimization problems. As such, they aim to find the single best solution. While evolutionary algorithms have achieved success in some areas, some argue that the true virtue of evolution is not optimization, but rather *divergence*, producing many different organisms that fill many niches [5].

As an example of a divergent evolutionary algorithm, *novelty search* seeks not for fitness but for phenotypic diversity [7]. By explicitly optimizing for phenotypic diversity, the evolution is able to constantly find novel solutions resulting in a uniform search in the phenotypic space. This can be beneficial for deceptive domains, where simple optimization of fitness leads to a dead end. This idea is similar to niching methods [8], but niching only guarantees genotypic diversity and does not guarantee phenotypic diversity.

The novelty of an individual is defined by how unique the phenotype of an individual is. This is commonly calculated as the sparseness ρ of a point x , equivalent to the average distance to its k -nearest neighbors:

$$\rho(x) = \frac{1}{k} \sum_{i=1}^k d(\mathbf{x}, \mu_i), \quad (1)$$

where μ_i is the i th-nearest neighbor of $\mathbf{x}$, using the distance met-

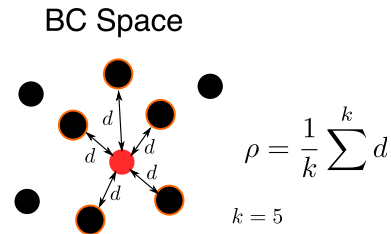


Fig. 1 Calculation of novelty. Novelty is calculated as the mean distance to k -nearest neighbors in the BC space.

ric d . This is illustrated in **Fig. 1**. This sparseness is calculated using the data expression of the phenotype, commonly referred to as *behavioral characterization* (BC). The BC should describe important features of an individual's phenotype as a compact vector.

2.1.2 Quality-diversity Algorithms

Advances in novelty search have led to the proposal of *quality-diversity* (QD) algorithm [9], which aims to seek both diverse and high-performing solutions. A simple method would be to use MOGA to optimize both fitness and novelty measures. This has been referred to as novelty search with global competition (NS-GC).

A variant of NS-GC replaces the global fitness measure that is shared by all individuals across all generations with local competition score. Local competition score is defined by how superior an individual is with its k -nearest neighbors in the BC space. This encourages competition to occur locally within every niche. As such, this method is referred to as novelty search with local competition (NS-LC) [10].

Another common QD algorithm is Map-Elites [11]. Map-Elites achieves quality-diversity by explicitly defining niches by partitioning the BC space. This encourages competition to occur within every niche, and high-performing individuals in every niche can be obtained.

In some cases, QD algorithms have been shown to outperform a fitness-based search in terms of the single best solution found [12]. This can be attributed to the fact that QD algorithms can generate stepping stones that lead to better solutions, by maintaining phenotypic diversity in the population [13].

QD algorithms have been applied to many areas like robot behavior learning [14], [15], [16] and industrial design [17]. There have also been applications to creative domains, most notably procedural content generation. For example, Map-Elites has been applied to Mario-like platformer game stage design [18], [19].

2.2 Sound Matching

Sound matching refers to the estimation of synthesis parameters that reproduce a given sound (**Fig. 2**). One motivation for sound matching is to assist the use of a synthesizer during sound design. The user can provide a sample sound that is close to their intention, and the result of sound matching can be used as an effective starting point for further sound design. Since users can learn how to reproduce a sound, sound matching is also effective as a teaching tool for sound design.

Another merit of sound matching is the *creative imitation* of a sound. For example, a trumpet sound reproduced with a synthe-

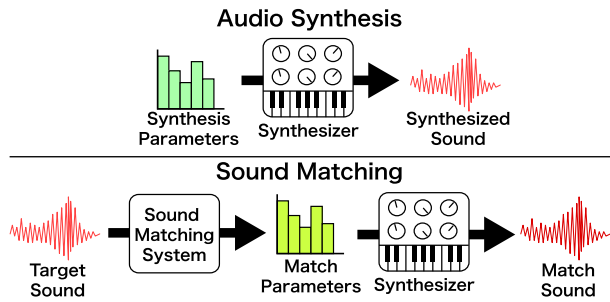


Fig. 2 Diagram of audio synthesis and sound matching. Sound matching attempts to solve the inverse problem of audio synthesis.

sizer will not sound the same as the original due to the constraints of the synthesizer. Furthermore, a trumpet sound reproduced by one synthesizer will sound different from that of another synthesizer, since each synthesizer has a unique timbre. By producing matches with different synthesizers, users can discover unique variants of the original sound.

2.2.1 Iterative Sound Matching

In the iterative approach for sound matching, the quality of the matches are evaluated and then used to guide the generation of the next matches. While simple algorithms like hill climbing has been used as well [2], most works have adopted GA-based approaches. One of the earliest works in sound matching methods used GA to estimate the parameters of FM synthesis to match a target tone [20]. The fitness is defined as the distance from the produced spectrum to the target spectrum.

Subsequent works have focused on more practical applications, matching an entire sound whose spectra vary over time. Many synthesizer architectures have been used for sound matching, such as a physical modeling synthesizer [21], FM synthesizer [2], and a complex synthesizer with many synthesis algorithms [22].

The main disadvantage of iterative sound matching methods is that it takes quite a long time to match a single sound. For example, in a case where multi-objective GA was used to optimize a complex synthesizer, each sound took 5 hours to match [22].

2.2.2 Model-based Sound Matching

In recent years, machine learning models such as linear regression [1] and neural networks [2], [3], [4] have been employed for the sound matching task. These models are trained to predict the synthesis parameter given a sound as input, solving an inverse problem of sound synthesis.

Such models can match sounds quickly (typically a few seconds) once the models are trained, but the models have to be trained every time there is a change in the layout of the synthesis modules. For example, in the case of modular synthesizers, a certain knob of a module may have a completely different meaning depending on the connection of the synthesis modules.

Another problem is that most methods are trained to minimize the error of the estimated parameter value. It has been noted that models with the least parameter error do not always return the closest match in terms of sound similarity [4], [23]. GA methods optimize the match in terms of sound similarity, but model-based methods minimize only the parameter loss and do not guarantee actual sound similarity. Also, this means that out-of-domain sounds (sounds not synthesized by the synthesizer) cannot be

used during training, as the ground-truth synthesis parameters that replicate them are unknown.

These problems can be solved by implementing a synthesizer with differentiable modules and backpropagating a spectral loss through the synthesizer [23]. However this approach has not yet been extended to complex synthesizers with features such as FM synthesis.

3. Proposed Method

In the proposed system, quality-diversity algorithms are used in iterative sound matching. In Section 3.1, the implementation of the baseline sound matching GA is explained. In Section 3.2, we will look at how this baseline GA is modified to be a quality-diversity algorithm.

3.1 Sound Matching Algorithm

In a GA-based sound matching system, the synthesis parameters correspond to the genotype of an individual, and the resulting output audio corresponds to the phenotype. First, synthesis parameters are randomly sampled to generate the initial population. The phenotypes of the population are obtained by rendering audio with the given synthesis parameters. Then, each phenotype is compared with the target audio to calculate the fitness of the individual. Then, individuals are selected, according to their fitness values. Finally, genetic operators such as crossover and mutation are applied to the genotype to create the new generation.

For the fitness of individuals, log-spectral distortion (LSD) is used. For target spectrum P and match spectrum $\hat{P}$, the log-spectral distortion can be calculated as

$$LSD(P, \hat{P}) = \sqrt{\sum_{\omega} 10 \log_{10} \left(\frac{P(\omega)}{\hat{P}(\omega)} \right)^2}, \quad (2)$$

3.2 Quality-diversity Algorithm

We introduce a novelty objective to GA-based sound matching to create a quality-diversity sound matching algorithm. We implement two quality-diversity algorithms: novelty search with global competition (NS-GC) and novelty search with local competition (NS-LC). While NS-GC optimizes both the match fitness (LSD between the target and output) and novelty, NS-LC replaces this with the local competition score, which is defined as the number of neighboring individuals in the BC space that is inferior in terms of fitness.

For optimizing two objectives, NSGA-II [24] was used. NSGA-II is a popular multi-objective GA method that speeds up the search for non-dominated individuals by reducing the computational complexity of non-dominated sorting and incorporating elitism.

The pseudocode for the quality-diversity sound matching algorithm is shown in Algorithm 1. Evaluation of individuals is performed by first synthesizing audio with the genotype of the individual i.e., synthesis parameters. This audio is used then to calculate the fitness and behavioral characteristics of the individual. Next, novelty is calculated as the mean distance to the k nearest neighbors. K nearest neighbors for an individual are searched from other individuals in the current population and the archive.

Algorithm 1 Proposed method (fitness & novelty)

Require: Target, Synthesizer

Initialize Population as list of random Individuals

Initialize Archive as empty list

for $gen = 1, \dots, N_{gen}$ **do**

for $ind \in \text{Population}$ **do** ▷ Evaluation of Individuals

$audio \leftarrow \text{Synthesizer}(ind.gtype)$

$ind.fitness \leftarrow -LSD(\text{Target}, audio)$

$ind.bc \leftarrow BC(audio)$

for $ind \in \text{Population}$ **do** ▷ Novelty calculation

$distances \leftarrow kNN(ind.bc, \text{Population} \cup \text{Archive})$

$ind.novelty \leftarrow \frac{1}{k} \sum distances$

$\text{Offspring} \leftarrow \text{SelectionNSGAI}(\text{Population})$

for $i = 0, \dots, N_{pop}/2 - 1$ **do** ▷ Genetic operators

$ind_a, ind_b \leftarrow \text{Offspring}[2i], \text{Offspring}[2i + 1]$

Mutate ind_a, ind_b at mutation rate p_{mut}

Crossover ind_a and ind_b at crossover rate p_{cx}

$\text{Population} \leftarrow \text{Offspring}$

Add λ Individuals from Offspring to Archive at random

Then, selection and genetic operators are applied following the conventions of NSGA-II. Finally, individuals are randomly added to the archive. It has been shown that this strategy for constructing the archive is just as effective as adding individuals that have the highest novelty score [25].

For the behavioral characterization (BC), two simple audio features were used: spectral centroid and spectral flatness. Spectral centroid is defined as the center of mass of a spectrum and has been shown to correlate with the “brightness” of a sound [26]. We convert the spectral centroid using the mel scale so that it aligns with human perception of pitch. Spectral flatness is defined as the ratio of the geometric mean to the arithmetic mean of the power spectrum, and describes how noise-like a spectrum is [27]. A spectrum with high flatness is either noisy or has many harmonics, and a spectrum with low flatness is close to a pure tone. For both features, the weighted mean over time according to the energy of each frame was used.

4. Experiment Setup

In the experiments, we performed sound matching and evaluated the quality and diversity of the produced matches. Sound match examples and the source code for the project including the synthesizers are available on the accompanying website^{*1}.

4.1 Sound Matching Algorithm

To examine the effectiveness of quality-diversity algorithms, we compared NS-GC and NS-LC against a GA baseline. For NS-GC and NS-LC, NSGA-II selection was used. For GA, tournament selection with tournament size of 2 is used to align with the NSGA-II selection. In all algorithms, elite selection was used to save 10 unique individuals with the best match quality and add to the next generation. The population size was set to 200.

All synthesizer parameters were continuous values within 0 ~ 1. While previous work in GA-based sound matching discretized the continuous synthesis parameters to use binary-coded GA [22], our method does not discretize the genotype. Instead, we use

real-valued genetic operators that have been shown to be more effective in continuous domains than discretized methods [28]. For mutation, polynomial mutation was used with a mutation rate $p_{mut} = 0.05$. For crossover, simulated binary crossover was used with a crossover probability $p_{cx} = 0.75$. The crowding degrees of both operators were set to 5.0. These parameters were adjusted to maximize the match quality of the GA baseline.

For the calculation of novelty, the mean distance between $k = 15$ neighbors were used. $\lambda = 5$ individuals were added every generation. Also, the behavioral characterization was bounded to restrict the area of search. The spectral centroid was bounded to be within the range of 400~1,600 mels (300~3,000 Hz). The spectral flatness was bounded to be within the range of -80~-30 dBs.

We also implemented a neural network regression model for sound matching (denoted as NN). This network predicts the synthesis parameter values from the spectrogram of the target sound. The network architecture was based on [3], but padding and stride sizes were changed to suit the audio length and batch normalization was added to prevent overfitting. The network was trained with L1 loss of the estimated synthesis parameters. The dataset used to train this model contained 200,000 randomly generated synthesizer outputs with 2,000 sounds set aside for validation. The network was trained for 100 epochs at a learning rate of 0.0005 with the Adam optimizer [29].

4.2 Synthesizers

For use in the sound matching experiment, two synthesizers were implemented in Faust programming language [30]. Previous works [2], [4], [6], [31] in sound matching using VSTs were difficult to replicate due to the nature of VST parameters, and some software synthesizers used were proprietary. The Faust code of the synthesizers used in our experiments was included in the repository on the accompanying website.

The synthesizers were played using DawDreamer [32], a python package that allows hosting and recording of VST and Faust instruments. The input to the synthesizer was a MIDI note at a pitch of C4 lasting 3 seconds. The output audio was recorded at a sample rate of 16 kHz for 4 seconds to capture the release of the note.

4.2.1 Subtractive Synthesizer

The first synthesizer we used is a subtractive synthesizer that is similar to a typical analog synthesizer. It roughly mimics features of Diva^{*2} used in previous works [4], [6], [31]. This synthesizer has three oscillators whose frequencies, amplitudes and waveform can be controlled. The first oscillator can modulate the frequencies of the second and third oscillators, and the modulation amounts can be controlled. The three oscillators and a white noise generator were then mixed and fed into a resonant low pass filter. The mixed audio amplitude and the filter frequency can each be controlled by an envelope and an LFO. The output of the filter was then fed into an effect section consisting of a flanger and a reverb connected serially.

4.2.2 FM Synthesizer

The second synthesizer we used is a frequency modulation

^{*1} <https://hyakuchiki.github.io/QDSynth/>

^{*2} <https://u-he.com/products/diva/>

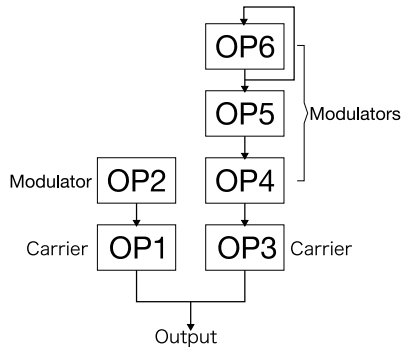


Fig. 3 Operator layout of FM synthesizer with ALGORITHM 1. Modulators can themselves be modulated by another operator.

(FM) synthesizer that emulates the Yamaha DX7. A basic FM synthesis with two sine oscillators can be described as

$$y(t) = \sin(2\pi f_c t + I \sin(2\pi f_m t)), \quad (3)$$

where f_c, f_m are the frequencies of the carrier and the modulator oscillator, and I is the modulation index [33]. FM is known to introduce inharmonic partials, which makes it suitable for creating a complex, rich timbre. The frequencies of the partials are determined by the frequencies of modulator and carrier oscillators and amplitudes are determined by Bessel functions.

We used a modified version of a DX7 emulation, the *dx7_ui* component, provided in the standard faust library. More specifically, the pitch modulation feature was removed, as we found that it usually did not help in creating better matches, but rather produced unorthodox sounds unsuitable for music production. The algorithm of the synthesizer, i.e., the operator layout was fixed to ALGORITHM 1 (shown in Fig. 3). As modulators of an oscillator can be modulated by another modulator, the relationship between the parameters and the output sound is complex, and makes the FM synthesizer more difficult to control than the subtractive synthesizer.

4.3 Test Sounds

To evaluate the sound matching performance, we prepared two test sets: *in-domain* and *out-of-domain*. Each test set consisted of 30 sounds.

The in-domain set consisted of sounds created using the same synthesizers triggered by the same MIDI note used for matching. Thus, it was theoretically possible to match the sounds perfectly, although a previous work has found that finding a close match is still challenging when using a complex synthesizer [2]. Since presets are not available for these synthesizers, synthesis parameter values were set to uniformly distributed random numbers and the output was recorded to create an entry in the in-domain set. While these sounds are not very representative of sounds actually used in music production, previous works use a similar data as a benchmark problem [2], [3].

The out-of-domain set consisted of musical sounds that would be used as a query to the system in real usage. This included sounds produced by a diverse range of acoustic and electronic instruments. These sounds were taken from the Nsynth Dataset [34]. The test set was extracted to be well spread out in terms of instrument type. These sounds were also in the same

pitch as the MIDI note used in the optimization.

4.4 Measures for Match Quality

While log-spectral distortion (LSD) was used as a measure of match quality during evolution, we additionally evaluated the final match quality in terms of mel-cepstral distortion (denoted as *MCD*) and loudness loss (denoted as *Loud*). *MCD* is a measure commonly used for measuring the quality of voice conversion and is calculated as the euclidean distance of mel-frequency cepstrum coefficients (MFCCs) [35]. When calculating the MFCCs, silent sections were trimmed from the audio, and the 1st MFCC which corresponds to signal power was discarded. The MFCC sequences were aligned using DTW before calculating their distance. Since MFCCs are known to correlate with timbre [36], *MCD* is expected to be a measure for how close the timbre of two sounds are. Also, loudness loss was calculated as the average L1 distance between the loudness of the target sound and the output sound at each spectral frame. Loudness was calculated using A-weighting with an FFT size of 2,048 and a frame rate of 50 Hz.

We also conducted subjective evaluation of the match quality with a simple preference test between two methods. 50 participants were gathered in total through a crowd-sourcing service and each participant answered 18 questions. In each question, participants were first presented with the target sound and then listened to matches by two methods. The presentation order of the matches was randomized. The target sounds were from the out-of-domain set, which is closer to queries in an actual sound matching system.

4.5 Measures for Quality-diversity

When evaluating QD algorithms, it is important to address not only the best solution found, but also the diversity of solutions found during the evolution. The goal of QD algorithm is to find high-performing solutions that are spread out in terms of the behavioral characteristics (BCs). For evaluation, it is common to partition the BC space in a similar manner to Map-Elites [11] and record the best individuals belonging to each area (commonly referred to as a *cell*) of the BC space. In this experiment, we partitioned the BC space evenly with 20 divisions in each dimension which results in 400 cells and recorded the best individuals found for each cell during every generation of the evolution. The resulting map is called the *feature-performance map* [11].

The most straightforward measure for QD is the *QD score*, calculated by adding up the fitness of best performing individuals from each occupied cell. A good QD algorithm would find high-performing solutions in many cells and thus have a high QD score. Performance was defined as the inverse of LSD between the target and the match.

A measure for diversity of a QD algorithm is *coverage*, calculated as the ratio of cells with at least a single individual occupying it. If the coverage is high, it means that the QD algorithm was able to find solutions from most areas of the BC space.

Another measure for evaluating the behavior of a QD algorithm is the *exploration uniformity* [25]. Let φ be the set of all individuals found during a run and I_i be the number of individuals assigned to i th cell. The exploration uniformity $U(\varphi)$ was

calculated using the Jensen-Shannon divergence between the distribution X_φ of all individuals and the uniform distribution Y in the BC space:

$$U(\phi) = 1 - JS D[X_\varphi \| Y], \quad (4)$$

where

$$X_\varphi = \left(\frac{I_1}{|\varphi|}, \dots, \frac{I_{400}}{|\varphi|} \right), \quad (5)$$

$$Y = \left(\frac{1}{400}, \dots, \frac{1}{400} \right). \quad (6)$$

A high value indicates that the algorithm explored uniformly in the BC space.

5. Results

5.1 Match Quality

The objective values of sound matching is shown in **Table 1** and **Table 2**. The tables show the best values for each measure among all individuals in the final population. For all measures, either standard GA or NS-GC found the best solutions. This suggests that optimization of novelty did not result in degradation of performance in terms of best match found. The results of the Wilcoxon signed rank test of the LSD values did not reveal any significant differences between GA and NS-GC. NN performed significantly poorly compared to GA-based methods, but it is worth noting that NN is able to find matches much quicker (few seconds) than GA-based methods (approximately 15 minutes). Thus, NN methods maybe useful for finding rough matches quickly.

The results of the subjective evaluation is shown in **Fig. 4**. For the subtractive synthesizer, NS-GC performed comparably to GA. For the FM synthesizer, GA was preferred over NS-GC. For both synthesizers, NS-LC performed significantly worse than GA and NS-GC.

5.2 Quality-diversity

We evaluated the quality-diversity of each algorithm in the case of out-of-domain sounds. Quality-diversity is evaluated by partitioning the BC space and calculating the measures we established in Section 4.5. We omitted the neural network-based method (NN) as it can only find one solution.

Table 1 Sound matching results for the Subtractive synthesizer.

	In-domain			Out-of-domain		
	LSD	MCD	Loud	LSD	MCD	Loud
GA	21.577	201.842	3.076	27.330	464.107	20.698
NS-GC	23.015	224.287	2.051	27.438	457.715	12.119
NS-LC	25.301	267.481	3.384	30.751	535.943	14.523
NN	36.123	299.773	2.701	56.090	786.270	30.297

Table 2 Sound matching results for the FM synthesizer.

	In-domain			Out-of-domain		
	LSD	MCD	Loud	LSD	MCD	Loud
GA	5.613	122.886	1.809	30.637	492.738	15.768
NS-GC	5.475	114.525	2.101	31.335	503.430	13.457
NS-LC	7.531	142.574	4.755	33.966	565.069	15.437
NN	22.164	334.495	10.609	68.291	1062.382	26.802

The QD Score is shown in **Fig. 5**. We found that NS-GC performed better than other methods in both synthesizers through a Wilcoxon signed rank test ($p < 0.01$). This result suggests that in terms of quality-diversity, NS-GC is the most effective algorithm.

The coverage is shown in **Fig. 6**. NS-GC and NS-LC had higher coverage compared to GA, indicating that the novelty objective was effective in covering a wide area of the BC space.

To gain insights into the dynamics of the evolution, we show the exploration uniformity of each algorithm in **Fig. 7**. NS-GC

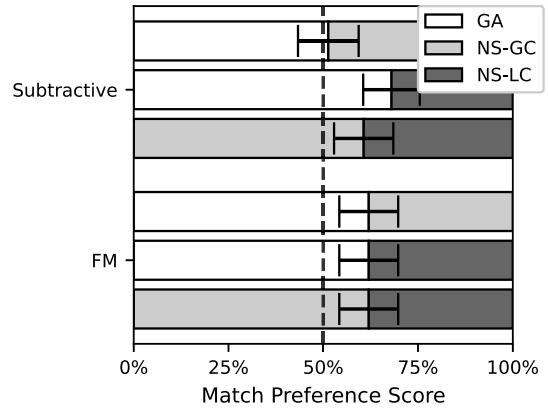


Fig. 4 Results of the subjective preference test. Error bars denote 95% confidence intervals.

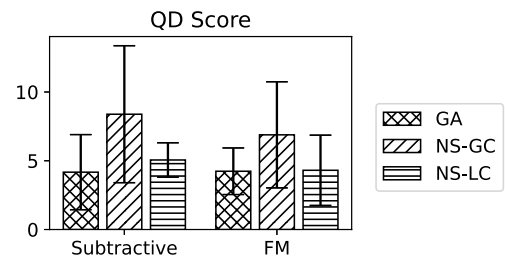


Fig. 5 Comparison of QD Score. Larger values indicate that the algorithm was able to find good matches across a wide region of the BC space. Error bars indicate standard deviation.

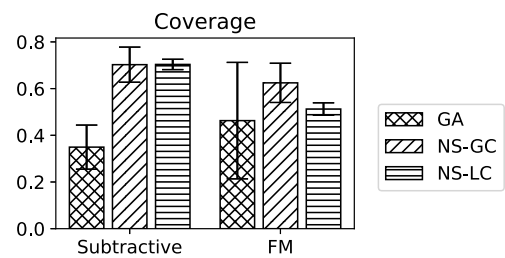


Fig. 6 Comparison of coverage of the evolution in the BC space. Values close to 1 indicate that the algorithm explored most of the cells in the BC space. Error bars indicate standard deviation.

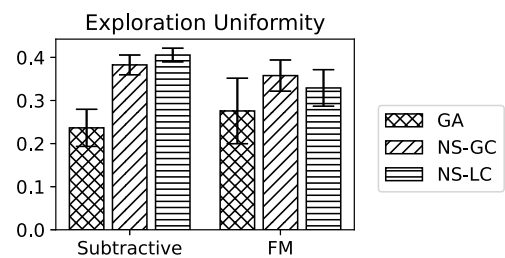


Fig. 7 Comparison of the exploration uniformity. Values close to 1 indicate that the individuals found during the evolution follow a uniform distribution in the BC space.

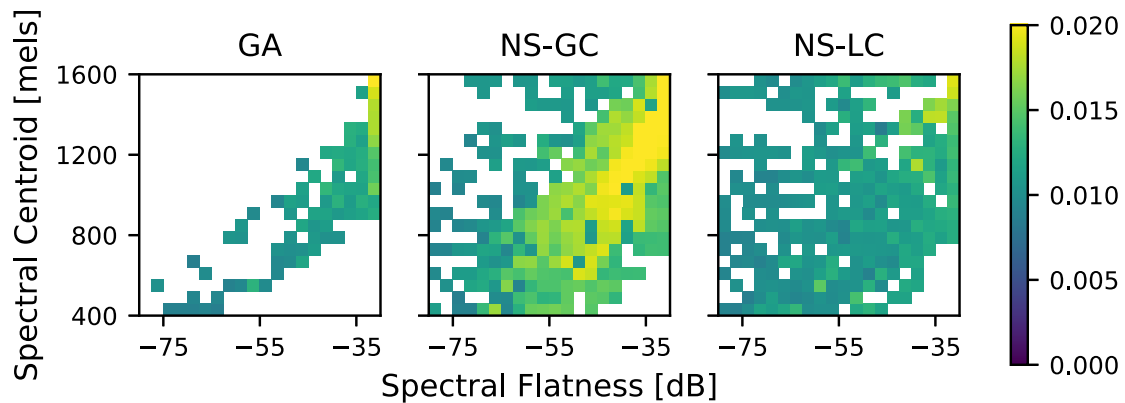


Fig. 8 Comparison of feature-performance map from a typical run. Lighter colors indicate that individuals with high performance (defined as inverse of LSD) were found in the cell. Bins with no assigned individual were colored white.

and NS-LC showed a higher exploration uniformity, indicating that the algorithm explored the BC space in a more even manner, while GA tended to focus more on certain regions.

Finally, we visualize the feature-performance map from a typical sound matching run with the subtractive synthesizer in **Fig. 8**. The target sound was an electronic synth lead from the NSynth dataset. QD methods tended to find fairly good matches across the BC space, while GA tended to find good matches only in a limited region.

6. Discussion

6.1 Exploration vs. Exploitation

As we showed in Fig. 7, QD methods explore the BC space more uniformly, while GA tended to exploit certain areas. We hypothesize that exploration is good for finding diverse matches, but some exploitation is needed for finding good matches globally. While GA was competitive in terms of finding a single good match, it did not explore enough to find diverse matches. NS-LC performed in a inferior manner to other methods in terms of global match quality. We suppose that NS-LC favored exploration over exploitation too much, and NS-GC was a good balance between exploration and exploitation.

6.2 Design of Behavioral Characteristics

Deciding the features to use for BCs is an important consideration when designing QD algorithms. We note that NS-GC and NS-LC tended to be worse in the case of the FM synthesizer compared to the subtractive synthesizer. We suppose that this is due to the mismatch between the FM synthesizer and the BC space. One dimension of the BC used in this experiment, spectral flatness, roughly corresponds to the noisiness of the sound. However, only the subtractive synthesizer has a noise generator. Upon listening to the matches produced by QD methods using the FM synthesizer, we noticed that the matches tended to produce noisy sounds by setting the FM parameters in an unorthodox way. These sounds often achieved low LSD values, but were sonically displeasing. This may be the reason for the QD methods performing in a inferior manner to GA in the case of FM synthesizer in subjective preference tests (Fig. 4).

Another concern is the *alignment* of the BCs with the match

quality. A BC is said to be highly aligned with the quality when each dimension of the BC is correlated with the fitness function and only certain regions can achieve high fitness [5]. The BCs in this sound matching experiment are fairly aligned with the match quality. When the BC is aligned, certain regions in the BC space tend to lead to higher fitness. The novelty objective can often help in finding such areas quickly by favoring exploration, but once such area is discovered, exploitation is effective for searching these areas extensively.

As a preliminary experiment, we explored the use of neural embeddings using models such as Wav2CLIP [37]. While the BC space defined by such models is useful for discovering novel sounds, the vast BC space defined by these models favors exploration and tended to inhibit exploitation.

6.3 Evaluation of QD

In Section 5.2, we evaluated the QD in terms of objective measures that are common in QD literature. The diversity is measured in terms of how far apart the solutions are in terms of the behavioral characteristics. Since the spectral features are known to correlate with perceptual qualities, this diversity measure is expected to correlate with the diversity as perceived by humans. It may also be insightful to evaluate this diversity through subjective dissimilarity ratings, but we leave this to future work.

We did not evaluate the QD of the neural network method since they only return a single result. While most neural network methods only return a single match, the variational autoencoder-based model proposed in Ref. [4] which learns a “macro control” is able to produce a match while providing the ability to create variants. However, the alignment of the learned macros with human perception has not been evaluated in depth, and adjustments of the macro may result in sounds that do not sound like the target. In comparison, our method is able to find variants that preserve the characteristics of the target sound.

6.4 Creative Applications of QD

It is possible to use the feature-performance map (Fig. 8) to find matches with a certain property. Regions with high spectral centroid and flatness had high match quality in this run, but we can look into other areas to find variants of this match. For exam-

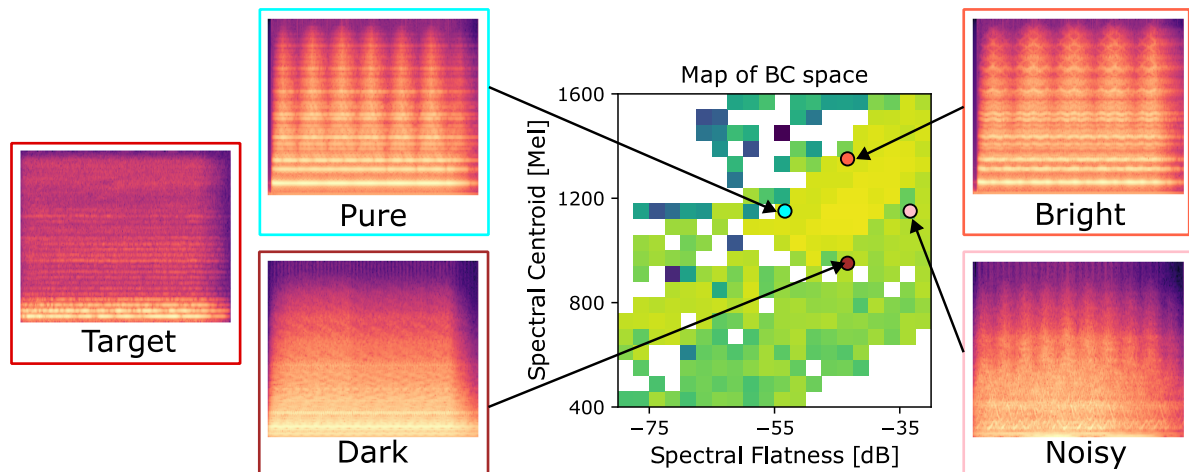


Fig. 9 Feature-performance map and the sounds it contains. The evolution used a novelty objective and a fitness based on loudness. The target sound was a choir sample from the NSynth dataset.

ple, we can look into areas with a lower spectral centroid to find matches with a “darker” quality.

It is also possible to relax the match constraint by only matching a certain property of the target sound. By doing so, we expect that the algorithm is able to explore more evenly across the BC space, because the BC is now less aligned with match quality. We display the feature-performance map of NS-GC sound matching with a loudness loss in **Fig. 9**. With this method, it is possible to obtain variations of a sound with certain characteristics. For example, the match with low spectral flatness (*Pure*) had less noise compared to the match with high spectral flatness (*Noisy*).

6.5 Running Time of the Evolution

One concern for the application of the proposed method is the time it takes to find a match. A single run of the sound matching algorithm of the subtractive synthesizer takes around 15 minutes, while the FM synthesizer takes approximately 25 minutes on a PC with Intel Core i7-6700 CPU. This is relatively fast compared to previous work on GA-based sound matching such as PresetGen that takes 5 hours to match a 2 seconds sound on a computing cluster [22]. Still, it is cumbersome for users to have to wait 15 minutes every time they want to find a match for a sound.

Profiling of the program revealed that approximately 90% of the running time is spent on audio synthesis. Thus, speeding up the synthesis is the key to speeding up the program. Parallelization of the synthesizer may be an effective solution. Recently, there has been work on differentiable synthesizers that process audio using the GPU. For example, torchsynth can synthesize audio 16,200 times faster than realtime [38]. While torchsynth supports only simple DSP elements, there has been work on extending GPU synthesis to synthesis algorithms such as FM [39].

7. Conclusion

In this paper, we introduced a QD approach to the sound matching problem by implementing a multi-objective genetic algorithm with novelty objective. The novelty of a match is characterized by spectral features, and the method allows for the discovery of matches that are diverse in terms of these features. We

evaluated the QD algorithms and baseline GA not only in terms of best match produced but also from a QD standpoint and showed the effectiveness of QD algorithms. This QD approach to sound matching may be helpful in assisting sound design and exploring the capabilities of a synthesizer.

References

- [1] Itoyama, K. and Okuno, H.G.: Parameter Estimation of Virtual Musical Instrument Synthesizers, *Proc. 40th Int. Computer Music Conf.*, No. September, pp.1426–1431 (2014).
- [2] Yee-King, M.J., Fedden, L. and D’Inverno, M.: Automatic Programming of VST Sound Synthesizers Using Deep Networks and Other Techniques, *IEEE Trans. Emerging Topics in Computational Intell.*, Vol.2, No.2, pp.150–159 (2018).
- [3] Barkan, O., Tsiris, D., Koenigstein, N. and Katz, O.: InverSynth: Deep Estimation of Synthesizer Parameter Configurations from Audio Signals, *IEEE/ACM Trans. Audio, Speech, and Lang. Process.*, Vol.27, No.11, pp.2385–2396 (2019).
- [4] Esling, P., Masuda, N., Bardet, A., Despres, R. and Chemla-Romeu-Santos, A.: Universal Audio Synthesizer Control with Normalizing Flows, *Proc. 22nd Int. Conf. Digital Audio Effects* (2019).
- [5] Pugh, J.K., Soros, L.B. and Stanley, K.O.: Quality Diversity: A New Frontier for Evolutionary Computation, *Frontiers Robotics AI*, Vol.3, No.40, pp.1–17 (2016).
- [6] Masuda, N. and Saito, D.: Quality Diversity for Synthesizer Sound Matching, *Proc. Int. Conf. Digit. Audio Effects* (2021).
- [7] Lehman, J. and Stanley, K.O.: Abandoning Objectives: Evolution Through the Search for Novelty Alone, *Evol. Comput.*, Vol.19, No.2, pp.189–223 (2011).
- [8] Mahfoud, S.W.: Niching Methods for Genetic Algorithms, PhD Thesis, University of Illinois at Urbana-Champaign (1995).
- [9] Pugh, J.K., Soros, L.B., Szerlip, P.A. and Stanley, K.O.: Confronting the Challenge of Quality Diversity, *Proc. Genetic and Evol. Comput. Conf.*, pp.967–974 (2015).
- [10] Lehman, J. and Stanley, K.O.: Evolving a Diversity of Creatures through Novelty Search and Local Competition, *Proc. Genetic and Evol. Comput. Conf.*, pp.211–218 (2011).
- [11] Mouret, J.-B. and Clune, J.: Illuminating Search Spaces by Mapping Elites, arXiv preprint arXiv:1504.04909 (2015).
- [12] Gaier, A., Asteroth, A. and Mouret, J.B.: Are Quality Diversity Algorithms Better at Generating Stepping Stones than Objective-based Search?, *Proc. Genetic and Evol. Comput. Conf. Companion*, pp.115–116 (2019).
- [13] Lehman, J., Stanley, K.O. and Miikkulainen, R.: Effective Diversity Maintenance in Deceptive Domains, *Proc. Genetic and Evol. Comput. Conf.*, pp.215–222 (2013).
- [14] Cully, A. and Mouret, J.-B.: Behavioral Repertoire Learning in Robotics, *Proc. Genetic and Evol. Comput. Conf.* (2013).
- [15] Cully, A., Clune, J., Tarapore, D. and Mouret, J.B.: Robots that can adapt like animals, *Nature*, Vol.521, No.7553, pp.503–507 (2015).
- [16] Duarte, M., Gomes, J., Oliveira, S.M. and Christensen, A.L.: Evolution of Repertoire-Based Control for Robots with Complex Locomotor Systems, *IEEE Trans. Evol. Comput.*, Vol.22, No.2, pp.314–328

- (2018).
- [17] Gaier, A., Asteroth, A. and Mouret, J.B.: Aerodynamic Design Exploration through Surrogate-assisted Illumination, *Proc. 18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conf.* (2017).
 - [18] Withington, O.: Illuminating Super Mario Bros: Quality-Diversity Within Platformer Level Generation, *Proc. Genetic and Evol. Comput. Conf. Companion*, pp.223–224 (2020).
 - [19] Fontaine, M.C., Liu, R., Khalifa, A., Togelius, J., Hoover, A.K. and Nikolaidis, S.: Illuminating Mario Scenes in the Latent Space of a Generative Adversarial Network, *Proc. 35th AAAI Conf. Artificial Intell.* (2020).
 - [20] Horner, A., Beauchamp, J. and Haken, L.: Machine Tongues XVI: Genetic Algorithms and Their Application to FM Matching Synthesis, *Computer Music Journal*, Vol.17, No.4, pp.17–29 (1993).
 - [21] Riionheimo, J. and Välimäki, V.: Parameter Estimation of a Plucked String Synthesis Model Using a Genetic Algorithm with Perceptual Fitness Calculation, *EURASIP Journal on Applied Signal Processing*, Vol.8, pp.791–805 (2003).
 - [22] Tatar, K., Macret, M. and Pasquier, P.: Automatic Synthesizer Preset Generation with PresetGen, *Journal of New Music Research*, Vol.45, No.2, pp.124–144 (2016).
 - [23] Masuda, N. and Saito, D.: Synthesizer Sound Matching with Differentiable DSP, *Proc. Int. Soc. for Music Information Retrieval Conf.* (2021).
 - [24] Deb, K., Pratap, A., Agarwal, S. and Meyarivan, T.: A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II, *IEEE Trans. Evolutionary Comput.*, Vol.6, No.2, pp.182–197 (online), DOI: 10.1109/4235.996017 (2002).
 - [25] Gomes, J., Mariano, P. and Christensen, A.L.: Devising Effective Novelty Search Algorithms: A Comprehensive Empirical Study, *Proc. Genetic and Evol. Comput. Conf.*, pp.943–950 (2015).
 - [26] Schubert, E.: Does Timbral Brightness Scale with Frequency and Spectral Centroid?, *Acta Acustica United with Acustica*, Vol.92, pp.820–825 (2006).
 - [27] Peeters, G.: A Large Set of Audio Features for Sound Description (Similarity and Classification) in the CUIDADO Project, Technical report (2004).
 - [28] Deb, K. and Agrawal, R.B.: Simulated Binary Crossover for Continuous Search Space, *Complex Systems*, Vol.9, pp.1–34 (online), DOI: 10.1.1.26.8485Cached (1994).
 - [29] Kingma, D.P. and Ba, J.: Adam: A Method for Stochastic Optimization, *Int. Conf. on Learning Representations* (2015).
 - [30] Orlarey, Y., Fober, D. and Letz, S.: FAUST: An Efficient Functional Approach to DSP Programming (2009).
 - [31] Le Vaillant, G., Dutoit, T. and Dekeyser, S.: Improving Synthesizer Programming From Variational Autoencoders Latent Space, *Proc. Int. Conf. Digit. Audio Effects* (2021).
 - [32] Braun, D.: DawDreamer: Bridging the Gap Between Digital Audio Workstation and Python Interfaces, *Proc. Int. Soc. for Music Information Retrieval Conf.* (2021).
 - [33] Chowning, J.M.: Synthesis of Complex Audio Spectra by Means of Frequency Modulation, *Journal of the Audio Engineering Soc.*, Vol.21, No.7, pp.526–534 (1973).
 - [34] Engel, J., Resnick, C., Roberts, A., Dieleman, S., Eck, D., Simonyan, K. and Norouzi, M.: Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders, *Proc. 34th Int. Conf. Machine Learning*, pp.1068–1077 (2017).
 - [35] Kominek, J., Schultz, T. and Black, A.W.: Synthesizer Voice Quality of New Languages, *The Workshop on Spoken Language Technologies for Under-Resourced Languages* (2008).
 - [36] Terasawa, H., Slaney, M. and Berger, J.: A Timbre Space for Speech, *Proc. 9th European Conf. Speech Communication and Technology*, pp.1729–1732 (2005).
 - [37] Wu, H.-H., Seetharaman, P., Kumar, K. and Bello, J.P.: Wav2CLIP: Learning Robust Audio Representations From CLIP, *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Process. (ICASSP)* (2022).
 - [38] Turian, J., Shier, J., Tzanetakis, G., McNally, K. and Henry, M.: One Billion Audio Sounds from GPU-enabled Modular Synthesis, *Proc. 23rd Int. Conf. Digital Audio Effects (DAFx2020)* (2021).
 - [39] Caspe, F., McPherson, A. and Sandler, M.: DDX7: Differentiable FM Synthesis of Musical Instrument Sounds, *Proc. 23rd Int. Soc. for Music Information Retrieval Conf.* (2022).



Naotake Masuda received his B.E. and M.S.I.T. from the University of Tokyo, Tokyo, Japan in 2018 and 2020, respectively. He is currently pursuing a doctor of engineering degree at the University of Tokyo. He was an intern at the Institute for Research and Coordination in Acoustics/Music (IRCAM) in Paris in 2019 during

his stay at Sorbonne University. He has worked on a number of projects on AI-assisted music production, and his current research interests include synthesizers, neural audio synthesis and signal processing. He received the IPSJ Yamashita SIG Research Award from the Information Processing Society of Japan in 2022.



Daisuke Saito received his B.E., M.S., and Dr. Eng. degrees from the University of Tokyo, Tokyo, Japan, in 2006, 2008, and 2011, respectively. From 2010 to 2011, he was a Research Fellow (DC2) of the Japan Society for the Promotion of Science. He is currently an associate professor in the Graduate School of Engineering,

the University of Tokyo. He is interested in various areas of speech engineering, including voice conversion, speech synthesis, acoustic analysis, speaker recognition, and speech recognition. Dr. Saito is a member of the International Speech Communication Association (ISCA), the Acoustical Society of Japan (ASJ), the Information Processing Society of Japan (IPSJ), the Institute of Electronics, Information and Communication Engineers (IEICE), and the Institute of Image Information and Television Engineers (ITE). He received the ISCA Award for the best student paper of INTERSPEECH 2011, the Awaya Award from the ASJ in 2012, and the Itakura Award from ASJ in 2014.